Double Linked List

A Double Linked List is a data structure where each node contains a value, a pointer to the next element, and a pointer to the previous element — enabling bidirectional navigation.

Node structure: [prev] ← Node(value) → [next]

Unlike single linked lists, each node knows both its predecessor and successor. In Scala 3, this is typically implemented using classes or case classes representing nodes.

Why Was It Created?

Double Linked Lists were designed to overcome the limitations of Single Linked Lists.

Single Linked List Problems

- Navigate forward only
- Removal requires knowing the previous node
- Mid-list operations are complex

Double Linked List Solutions

- Navigate forward and backward
- Removals simplified to updating two pointers
- More efficient mid-list operations

When to Use It

Double Linked Lists shine when you need flexibility in navigation or frequent structural modification.

Bidirectional Navigation

Browser history, undo/redo in text editors — move forward and backward with ease.

Frequent Insertions & Removals

Insert or remove mid-list in O(1) when the node is already known.

Building Other Structures

Foundation for LRU Cache, Deque, and some memory managers.

When *Not* to Use It

Despite its flexibility, the Double Linked List has real costs worth considering.

✕ High Memory Usage

Each node stores a value plus two pointers — more overhead than arrays or Single linked lists.

✕ Slow Index Access

Accessing element *i* requires traversing the list: O(n) vs. array's O(1).

✕ Poor Cache Locality

Nodes are scattered in memory, reducing CPU cache efficiency and overall performance.

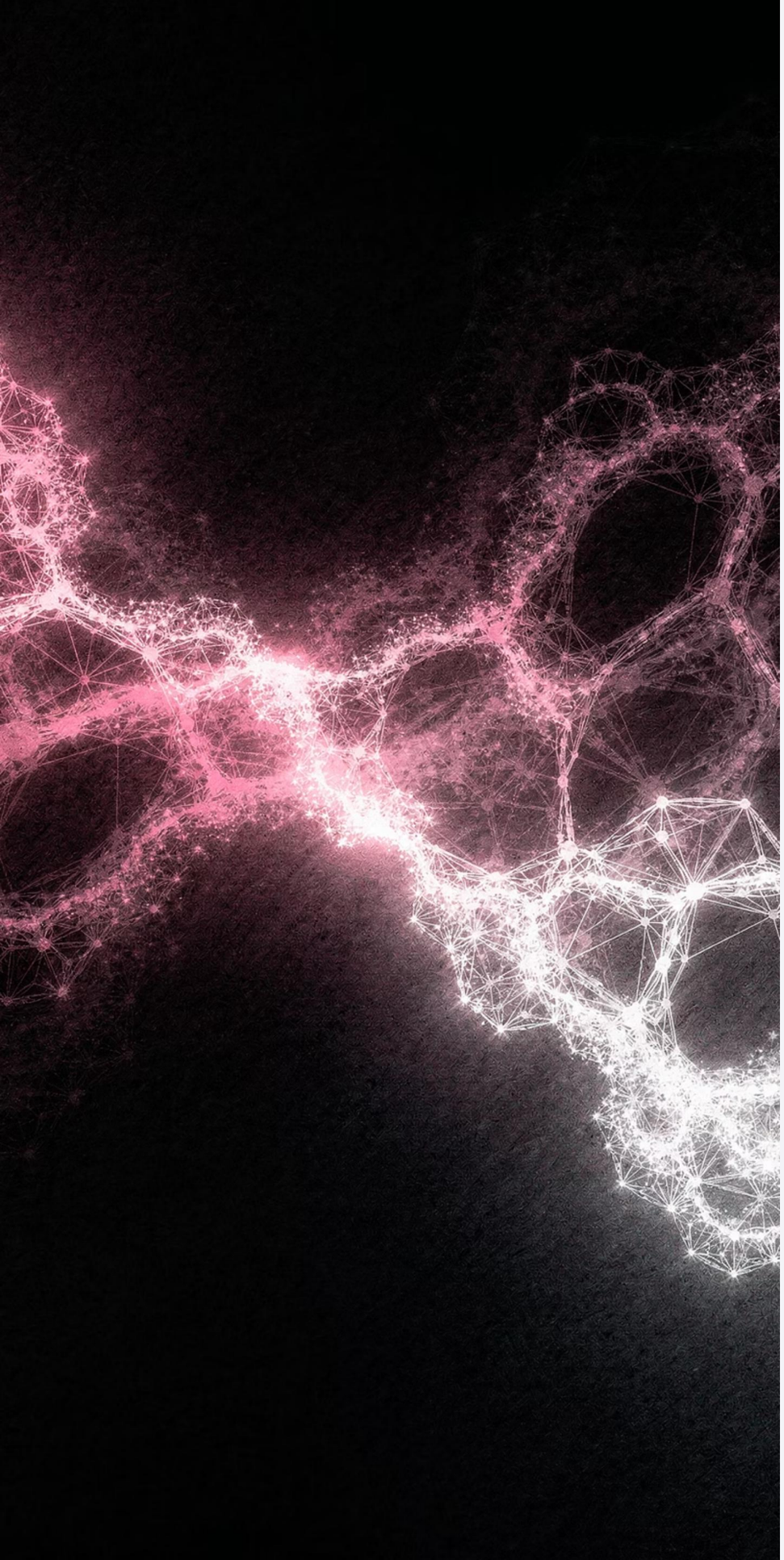
Trade-offs

✓ Advantages

- Bidirectional navigation
- Fast insertions & removals
- Dynamic, flexible structure
- Great for node manipulation

✗ Disadvantages

- Higher memory consumption
- Slow index-based access
- More complex implementation
- Less CPU cache-friendly



Conclusion

Double Linked Lists offer bidirectional navigation, efficient insertions/removals, and great structural flexibility — at the cost of higher memory use and slow index access.

✅ Use When

Frequent structural changes, bidirectional traversal, or building LRU Cache / Deque.

❌ Avoid When

Fast index access is required — prefer arrays instead.